

ANÁLISE COMPLETA DO PROJETO SISTEMA DE LAUDOS EEG COM IA

Avaliação de Arquitetura, Stack Tecnológica,
Performance, Escalabilidade e Visão de Mercado

Data: 13/04/2026

Versão 1.0

1. RESUMO DO ESCOPO ORIGINAL

O plano original propõe um MVP de 6 meses com 3 camadas tecnológicas distintas:

- **Frontend:** Angular 17 (PWA)
- **Backend:** .NET 8 Web API (Clean Architecture)
- **IA:** Python FastAPI (microserviço separado)

Orçamento estimado: ~R\$ 520.000 | Equipe: 5 pessoas | Meta: reduzir tempo de laudo de 20-30 min para 10 min por exame.

2. PROBLEMAS CRÍTICOS DA ARQUITETURA ORIGINAL

2.1 Complexidade desnecessária para um MVP

Problema	Impacto
3 linguagens/stacks (.NET + Angular + Python)	Requer 3 perfis diferentes, dificulta contratação e manutenção
Clean Architecture em .NET para CRUD simples	Over-engineering para um MVP de 6 meses
Angular 17 + NgRx + RxJS	Pesado demais; curva de aprendizado alta para equipe pequena
gRPC entre .NET e Python	Complexidade de integração sem benefício real nesta escala
Dois backends (.NET + FastAPI)	Duplicação de lógica de autenticação, validação, routing

2.2 Gargalos de performance identificados

- Retornar dados EEG como **JSON via REST** (19 canais × 256 Hz × 60 min ≈ 175 MB de JSON) é inviável
- **Chart.js / Plotly.js** não escalam para visualização contínua de 19 canais EEG em tempo real
- Ausência de **WebSocket** para streaming de dados do sinal
- Sem estratégia de **compressão** dos dados do sinal

2.3 Gaps técnicos do escopo

- Não menciona formato de exportação de dados (HL7 FHIR, DICOM-EEG)
- Sem plano de backup do modelo de IA (se a Claude API ficar offline)
- Sem estratégia de versionamento de modelos ML
- Sem pipeline de CI/CD para modelos (MLOps)
- Sem plano de migração de dados de sistemas legados
- Sem módulo de agendamento/workflow (fila de exames)

- Sem notificações (exame pronto, laudo pendente)
- Sem módulo de faturamento/relatórios gerenciais
- Sem integração com sistemas hospitalares (HIS/RIS)

3. ARQUITETURA PROPOSTA — CONSOLIDADA EM PYTHON

Eliminando .NET e Angular, consolidamos tudo em Python, ganhando:

- **1 linguagem principal** para toda a equipe
- **Contratação mais fácil** — Python é a linguagem mais popular do mundo
- **Ecossistema unificado** de IA/ML nativo
- **Menor custo de equipe** — 2-3 devs ao invés de 5

3.1 Stack Tecnológica Recomendada

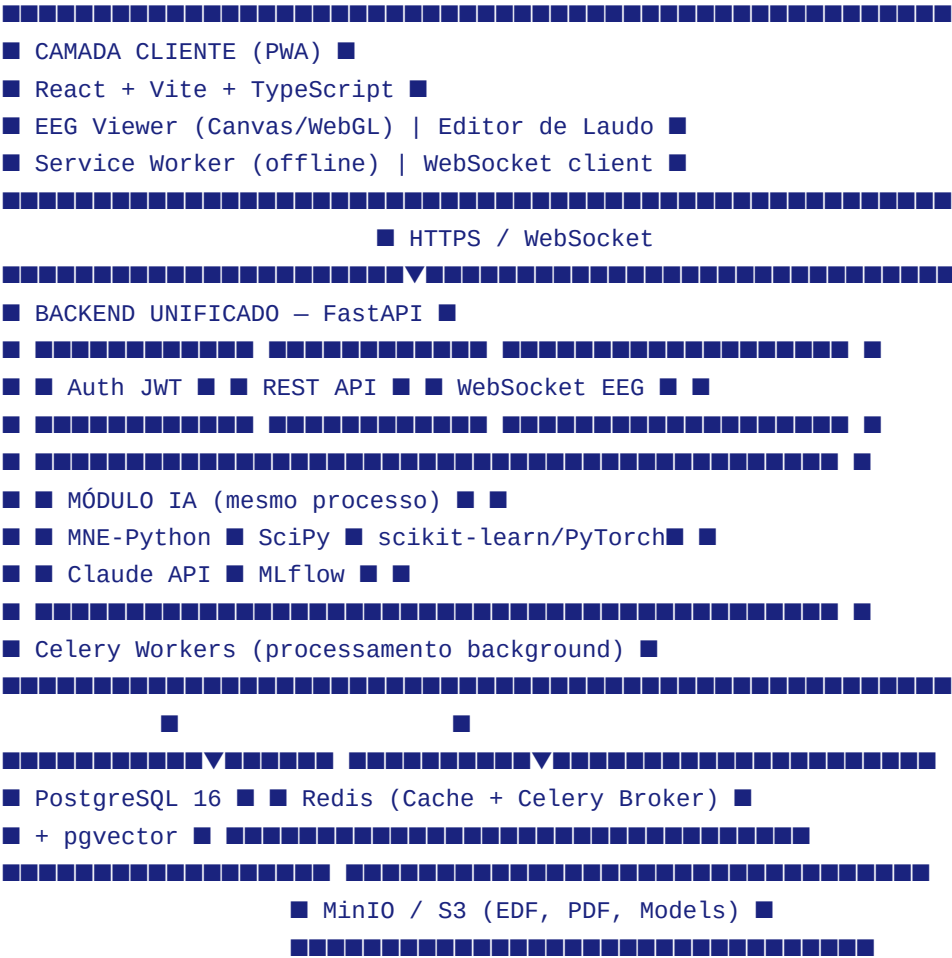
Camada	Tecnologia	Justificativa
Frontend (PWA)	React + Vite + TypeScript	Melhor ecossistema para Canvas/WebGL (EEG viewer). PWA nativo.
Backend API	FastAPI (async)	Alta performance, documentação automática, tipagem nativa, async I/O
Autenticação	fastapi-users + JWT + OAuth2	Solução completa, RBAC integrado
ORM	SQLAlchemy 2.0 + Alembic	Async support, migrações robustas
Tarefas assíncronas	Celery + Redis	Processamento de EEG em background
WebSocket	FastAPI WebSocket nativo	Streaming de dados EEG em tempo real
Geração PDF	WeasyPrint ou ReportLab	PDFs profissionais com layout customizado
Processamento EEG	MNE-Python + SciPy	Padrão da indústria para EEG
ML Clássico	scikit-learn	Baseline rápido (Random Forest)
Deep Learning	PyTorch	Evolução futura (CNN para detecção de spikes)
LLM	Anthropic SDK (Claude)	Geração de texto médico em português
MLOps	MLflow	Versionamento de modelos, tracking de experimentos
Banco de dados	PostgreSQL 16 + pgvector	Relacional + embeddings para busca semântica
Cache/Fila	Redis	Cache de sessão + broker Celery
Storage	MinIO ou AWS S3	Arquivos EDF, PDFs, modelos treinados
Containerização	Docker + Docker Compose	Ambiente reproduzível
CI/CD	GitHub Actions	Pipeline automatizado

3.2 Decisão de Frontend — Opções Ranqueadas

Opção	Prós	Contras	Recomendação
React + Vite (PWA)	Melhor para EEG viewer (WebGL/Canvas), PWA nativo, mercado amplo	Precisa de dev JS/TS	MELHOR PARA PRODUÇÃO

Opção	Prós	Contras	Recomendação
NiceGUI / Reflex	100% Python, rápido para MVP	Limitado para visualização EEG complexa	Bom para admin/dashboard
Streamlit	MVP em semanas	Não escala, sem PWA, sem controle fino de UI	Apenas protótipo
HTMX + Jinja2	Leve, Python puro no backend	Limitado para interatividade rica do EEG viewer	Viável exceto viewer

4. DIAGRAMA DA ARQUITETURA PROPOSTA



5. COMPARAÇÃO: ESCOPO ORIGINAL vs. PROPOSTA

Aspecto	Original (.NET + Angular + Python)	Proposto (Python + React)
Linguagens	3 (C#, TypeScript, Python)	2 (Python, TypeScript)
Backends	2 (.NET API + FastAPI)	1 (FastAPI)
Equipe mínima	5 pessoas	2-3 pessoas
Custo estimado	R\$ 520.000	R\$ 250.000 - 300.000
Time to market	6 meses	4-5 meses
Comunicação inter-serviços	HTTP/gRPC entre .NET e Python	Chamada direta de função
Deploy	3 containers distintos	1-2 containers
Autenticação	Duplicada em 2 backends	Única no FastAPI

Aspecto	Original (.NET + Angular + Python)	Proposto (Python + React)
ORM	Entity Framework (C#)	SQLAlchemy 2.0 (Python)
Performance IA	Latência de rede .NET → Python	Zero latência (mesmo processo)

6. VISÃO DE MERCADO

6.1 Concorrentes no Brasil e no mundo

Produto	País	O que faz	Preço aprox.
Persyst	EUA	Detecção automática de spikes em EEG	~\$500/mês/licença
BESA	Alemanha	Análise de sinais EEG/MEG	~€3.000 licença única
Natus NeuroWorks	EUA	EEG completo (hardware + software)	Bundled c/ hardware
EpiScan	Brasil (UFMG)	Detecção de crises epiléticas	Acadêmico
Neuroelectrics NIC2	Espanha	Análise EEG com IA	~€200/mês

6.2 Diferenciais do produto

- **IA + LLM para geração de laudo em português** — nenhum concorrente oferece isso
- **PWA offline** — funciona em clínicas com internet instável
- **Preço competitivo** para mercado brasileiro (R\$ 500/mês vs \$500/mês dos importados)
- **Aprendizado contínuo** com feedback médico (active learning)

6.3 Oportunidades de mercado

- ~5.000 clínicas de neurofisiologia no Brasil
- SUS realiza ~300.000 EEGs/ano → potencial de licitação pública
- Telemedicina: laudos a distância é tendência pós-COVID
- Expansão futura: polissonografia, EMG, potenciais evocados (mesma base tecnológica)

7. ITENS NECESSÁRIOS PARA INICIAR O DESENVOLVIMENTO

7.1 Itens obrigatórios ANTES de codar

#	Item	Status	Ação necessária
1	Arquivo EDF de exemplo	FALTA	Pedir ao médico ao menos 1 arquivo .EDF real (anonimizado)
2	Modelo de laudo padrão	OK	O PDF do Isaac serve como template
3	Definição de montagens EEG	PARCIAL	Confirmar com médico quais montagens usar
4	Lista de padrões anormais	PARCIAL	Falta: polipontas, surto-supressão, PLED, GRDA etc.

#	Item	Status	Ação necessária
5	Requisitos ANVISA	FALTA	Consultar se precisa registro como SaMD
6	Consentimento LGPD	FALTA	Modelo de termo de consentimento
7	Ambiente de desenvolvimento	PARCIAL	Faltam: PostgreSQL, Redis, Node.js
8	Credenciais Claude API	FALTA	Criar conta Anthropic e obter API key
9	Dataset público de EEG	DISPONÍVEL	TUH EEG Corpus (Temple University) — gratuito
10	Domínio e SSL	FALTA	Para deploy futuro

7.2 Perguntas pendentes para o stakeholder/médico

1. Quantos tipos de exame o sistema vai suportar inicialmente? (Só EEG rotina? Vídeo-EEG? Polissonografia?)
2. Quantos médicos vão usar o sistema na fase inicial?
3. O sistema precisa gerar nota fiscal ou integrar com sistema de faturamento?
4. Qual a meta de laudos/dia? (Isso define a infraestrutura necessária)
5. Já existe algum sistema legado que precisa de migração de dados?
6. O médico aceita usar o sistema para marcar exames durante o desenvolvimento (treino da IA)?

8. ESTRUTURA DE PROJETO PROPOSTA

```
eeg-laudo-system/
├── backend/                                # FastAPI
│   ├── app/
│   │   ├── api/                          # Rotas (auth, patients, exams, reports)
│   │   ├── core/                         # Config, security, database
│   │   ├── models/                       # SQLAlchemy models
│   │   ├── schemas/                     # Pydantic schemas
│   │   ├── services/                    # Business logic
│   │   ├── ml/                          # Módulo IA
│   │   │   ├── preprocessing/          # Filtros, segmentação
│   │   │   ├── detectors/              # Spike detector, asymmetry, base rhythm
│   │   │   ├── report_gen/             # Geração de laudo (Claude API)
│   │   │   └── models/                 # Modelos treinados (.pkl, .pt)
│   │   ├── migrations/                 # Alembic
│   │   ├── tests/
│   │   └── pyproject.toml
│   └── frontend/                        # React + Vite (PWA)
│       ├── src/
│       │   ├── components/
│       │   │   ├── EEGViewer/          # Canvas/WebGL renderer
│       │   │   ├── ReportEditor/       # Editor de laudo (rich text)
│       │   │   └── MarkingTool/        # Marcação de eventos
│       │   ├── pages/
│       │   ├── services/               # API client
│       │   ├── stores/                 # Zustand (estado)
│       │   └── package.json
│       ├── docker-compose.yml
│       ├── .env.example
│       └── README.md
```

9. VEREDICTO FINAL

O escopo original é **bem elaborado clinicamente** mas **over-engineered tecnicamente** para um MVP. A consolidação em Python + React proporcionará:

Benefício	Resultado
Redução de custo	~40% (de R\$ 520k para ~R\$ 280k)
Acелeração	~30% (de 6 para 4-5 meses)
Simplificação	1 backend ao invés de 2
Iteração com médico	Mais rápida com menos camadas de complexidade

Item mais crítico: O arquivo .EDF real é indispensável para iniciar o desenvolvimento do pipeline de processamento. Sem ele, todo o trabalho de IA fica bloqueado.

ROI Esperado (mantido do escopo original):

- Investimento revisado: ~R\$ 280.000
- Por cliente (R\$ 500/mês): 100 clientes = R\$ 50.000/mês
- Break-even em ~6 meses (antes era ~11 meses)
- Ano 2: R\$ 600.000/ano de receita